INFORMATIK LK · WELT 5 · DS 31–36

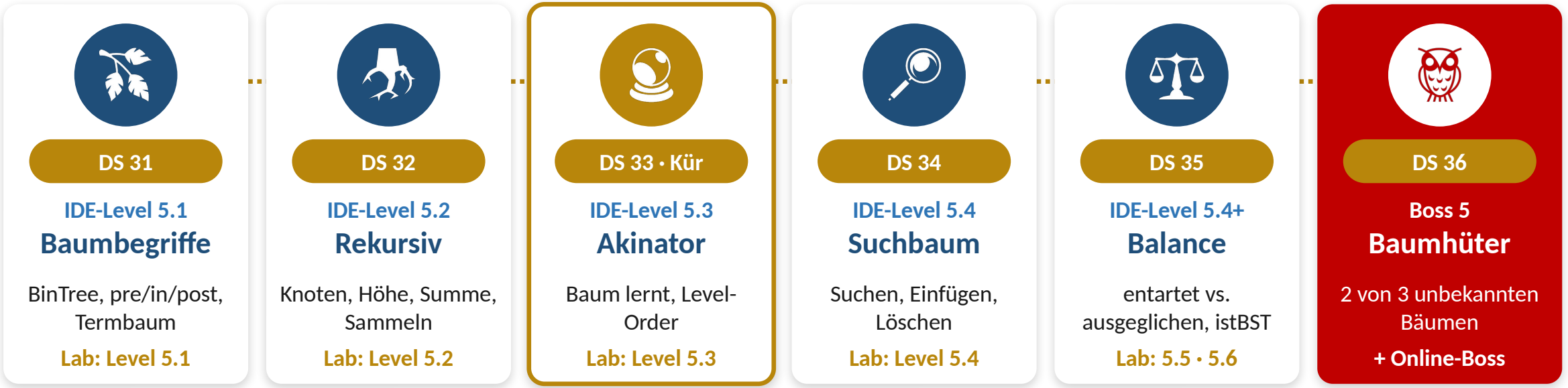
Der Wald

Binärbäume, binäre Suchbäume, Transfer



L5.1 · L5.2 · **L5.3** · L5.4 · L5.4+ · BOSS

Weltkarte: Der Wald



Am Ende von Welt 5 kannst du …

- Bäume mit Fachbegriffen beschreiben und mit BinTree aufbauen
- pre-, in-, post- und level-order von Hand angeben
- rekursive Baumalgorithmen schreiben – auch als Struktogramm
- im Suchbaum suchen, einfügen, löschen und den Aufwand begründen
- eine unbekannte Baumart aus Material erschließen

Algo Quest Lab in Welt 5

Level 5.1–5.6: automatisch geprüft
Punkte: beim ersten Lösen, automatisch
Trainingsplatz: Suchbaum, Traversierungen
Online-Boss: Die drei Baumhüter
Papier & IDE: Abi-Training ohne Punkte

Einstieg: Bäume wachsen überall

PAPIER · ohne Gerät · 10 min

Papier-Warm-up W5-01: Der Setzling

```
1  BinTree b = new BinTree(8);
2  b.setLeft(new BinTree(3));
3  b.setRight(new BinTree(10));
4  b.getLeft().setLeft(new BinTree(1));
5  b.getLeft().setRight(new BinTree(6));
6  b.getRight().getRight().setItem(14);
7  BinTree t = b.getLeft().getRight();
8  t.setLeft(new BinTree(4));
9  t.setRight(new BinTree(7));
10 b.getRight().getRight().setLeft(new BinTree(13));
```

a) Führe die Zeilen 1 bis 10 aus und zeichne den Baum b.

b) Gib Wurzel, Blätter, innere Knoten, die Höhe und die Ebene des Knotens 7 an.



Ordner und Dateien

Ein Ordner enthält Ordner – bis zur einzelnen Datei.



Turnierbaum

Zwei Spiele führen zu einem Sieger, bis zum Finale.

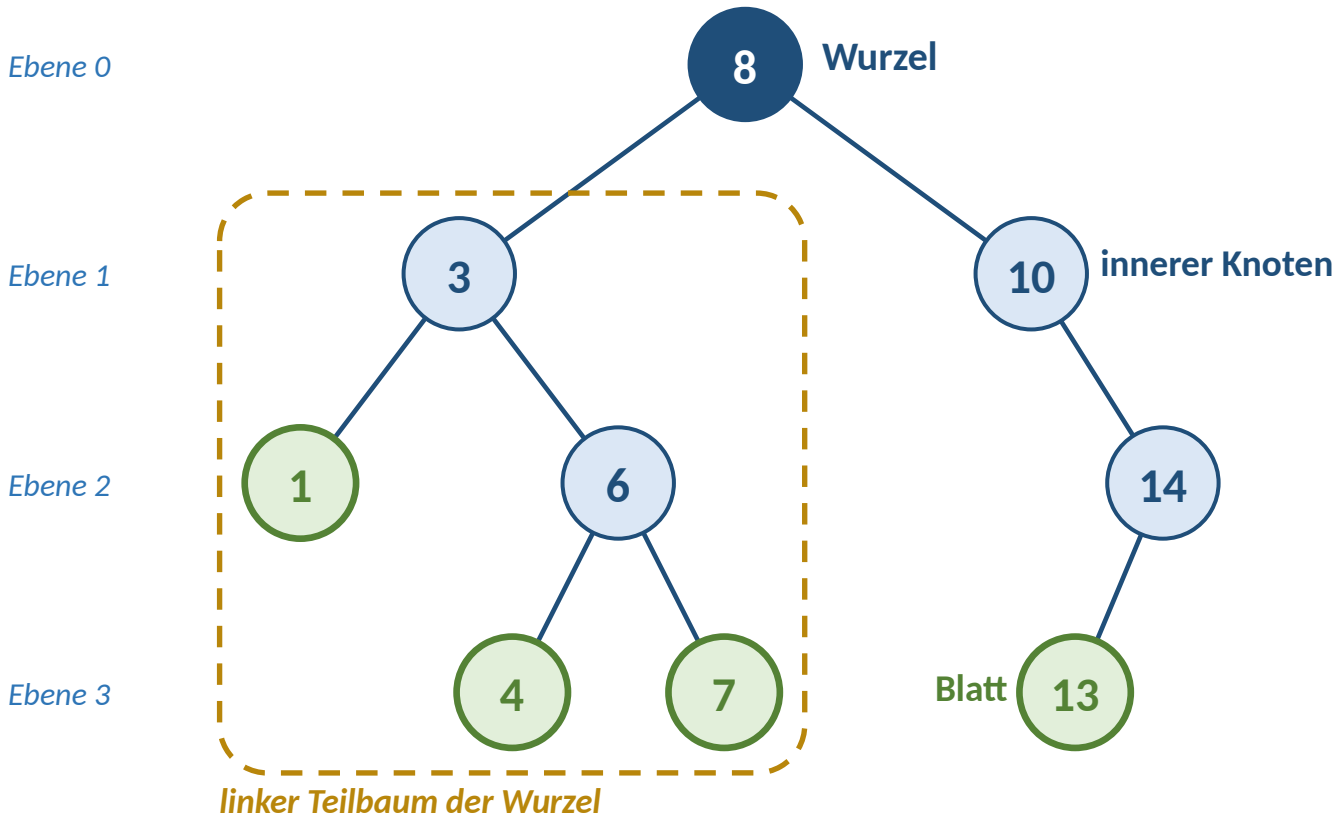


Entscheidungsbaum

Ja/Nein-Fragen führen zur Antwort – unser Akinator.

Leitfrage: Was haben diese Bilder gemeinsam? Oben ein Anfang, jede Verzweigung teilt das Problem – und unten ist Schluss.

Baumbegriffe am Beispiel beispielBST()



Wurzel	oberster Knoten (hier 8)
Blatt	Knoten ohne Kinder (1, 4, 7, 13)
innerer Knoten	Knoten mit mindestens einem Kind
Teilbaum	ein Kind mit allem darunter
Ebene	Abstand zur Wurzel (Wurzel: 0)
Höhe	Anzahl der Ebenen – hier 4

Höhe rekursiv:

```
leerer Baum → 0 · Blatt → 1
sonst 1 + max(links, rechts)
```

Achtung: Ebenen zählen ab 0, die Höhe zählt Ebenen – ein Blatt hat Höhe 1.

Knoten 9 · Blätter 4 · Höhe 4

Die Schnittstelle BinTree nach EH §4

Operation	Wirkung	leer?
`c BinTree()`	erzeugt einen leeren Baum	
`c BinTree(inhalt: Inhaltstyp)`	Blatt: Wurzel mit inhalt, zwei leere Teilbäume	
`isEmpty(): Wahrheitswert`	wahr, wenn der Baum leer ist	
`getItem(): Inhaltstyp`	Inhalt der Wurzel	**Fehler**
`setItem(inhalt: Inhaltstyp)`	setzt den Inhalt; leerer Baum wird zum Blatt	
`isLeaf(): Wahrheitswert`	wahr, wenn beide Teilbäume leer sind	
`getLeft(): Binärbaum`	linker Teilbaum	**Fehler**
`getRight(): Binärbaum`	rechter Teilbaum	**Fehler**
`setLeft(b: Binärbaum)`	setzt b als linken Teilbaum	**Fehler**
`setRight(b: Binärbaum)`	setzt b als rechten Teilbaum	**Fehler**
`setEmpty()`	macht den Baum zum leeren Baum	

In Signaturen: b: Binärbaum vom Inhaltstyp Ganzzahl

Nie null! Ein nicht-leerer Baum hat immer zwei Teilbäume – notfalls leere Bäume. Darum fragst du isEmpty(), nicht == null.

Casten wie in Welt 4

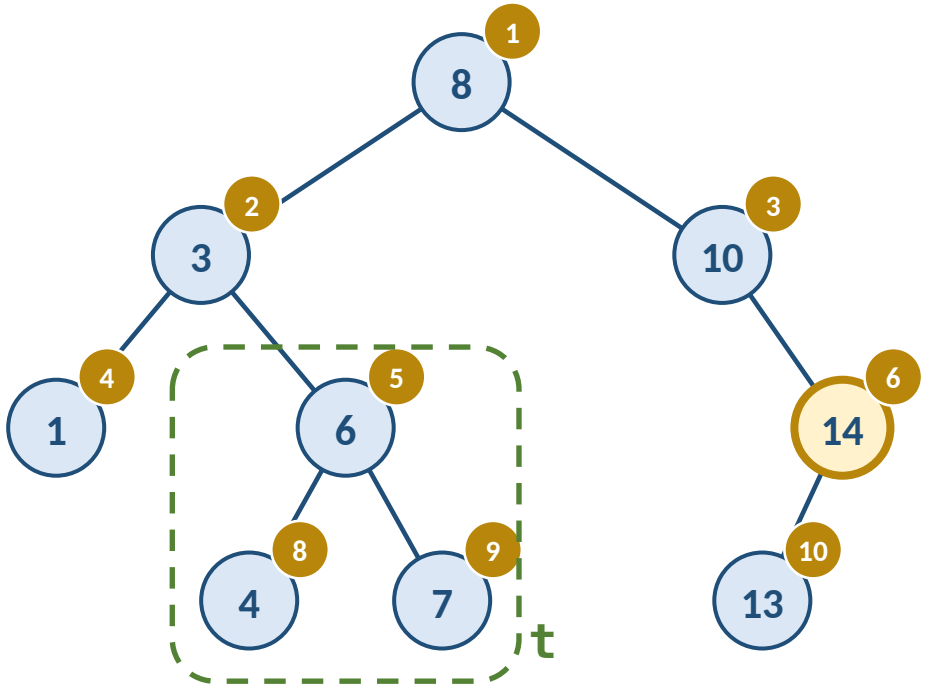
```
int x = (Integer) b.getItem();
String s = (String) b.getItem();
BinTree l = b.getLeft(); // ohne Cast
```

Leer? getItem, getLeft, getRight, setLeft, setRight auf einem leeren Baum → IllegalStateException. setItem(null) ist verboten.

Code ↔ Bild: Der Setzling wächst

```
1  BinTree b = new BinTree(8);
2  b.setLeft(new BinTree(3));
3  b.setRight(new BinTree(10));
4  b.getLeft().setLeft(new BinTree(1));
5  b.getLeft().setRight(new BinTree(6));
6  b.getRight().getRight().setItem(14);
7  BinTree t = b.getLeft().getRight();
8  t.setLeft(new BinTree(4));
9  t.setRight(new BinTree(7));
10 b.getRight().getRight().setLeft(new BinTree(13));

int x = (Integer) b.getLeft().getLeft().getLeft().getItem();
```



goldene Zahl = Codezeile, in der der Knoten entsteht

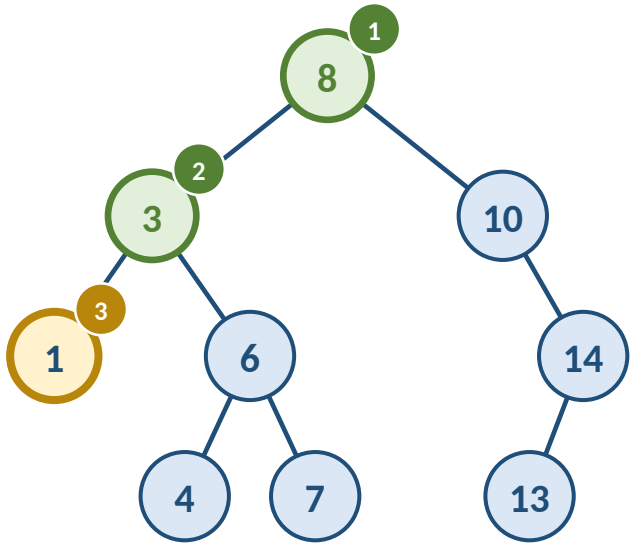
Zeile 6: Rechts unter 10 liegt ein leerer Baum, nicht null – setItem macht ihn zum Blatt 14.

Laufzeitfehler: Unter dem Blatt 1 liegt ein leerer Baum → getItem() → IllegalStateException.

Zeilen 7–9: t ist eine zweite Referenz auf den Teilbaum 6 – keine Kopie. Was t ändert, sieht auch b.

Traversierungen: pre-, in- und postorder (1/3)

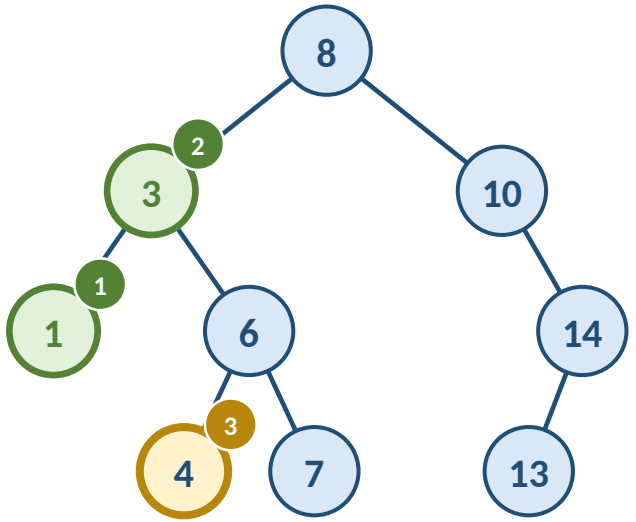
`preorder` Wurzel · links · rechts



Ausgabe: `"8 3 1 "`

rekursiv:
`item + " " + pre(l) + pre(r)`

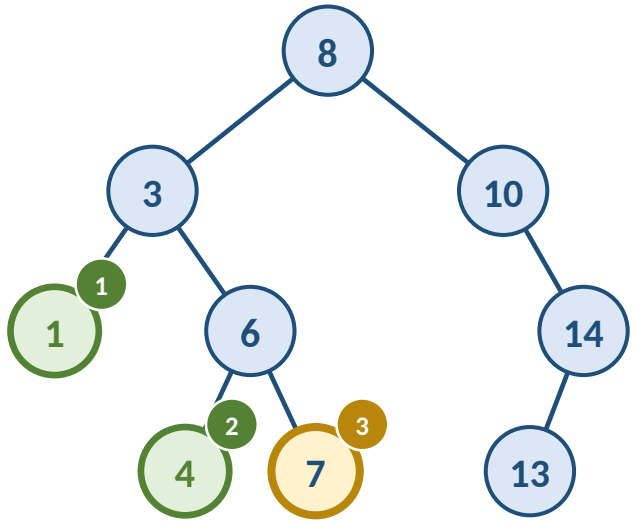
`inorder` links · Wurzel · rechts



Ausgabe: `"1 3 4 "`

rekursiv:
`in(l) + item + " " + in(r)`

`postorder` links · rechts · Wurzel

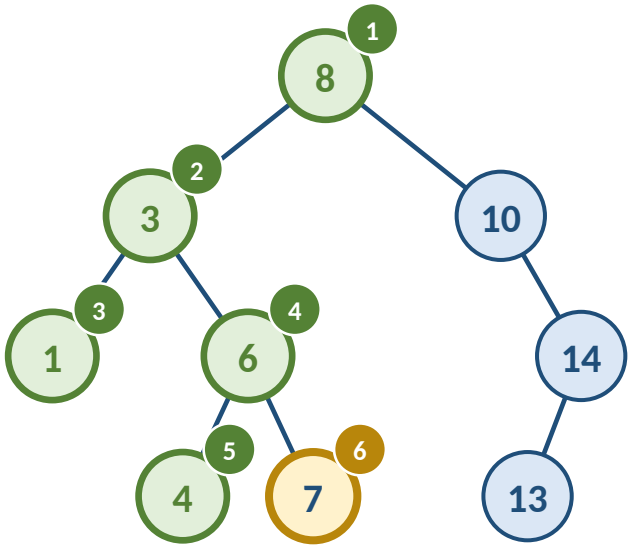


Ausgabe: `"1 4 7 "`

rekursiv:
`post(l) + post(r) + item + " "`

Traversierungen: pre-, in- und postorder (2/3)

`preorder` Wurzel · links · rechts

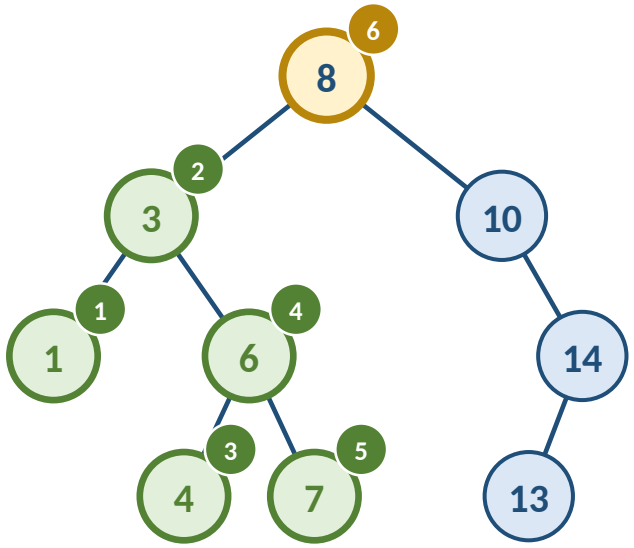


Ausgabe: `"8 3 1 6 4 7 "`

rekursiv:

```
item + " " + pre(l) + pre(r)
```

`inorder` links · Wurzel · rechts

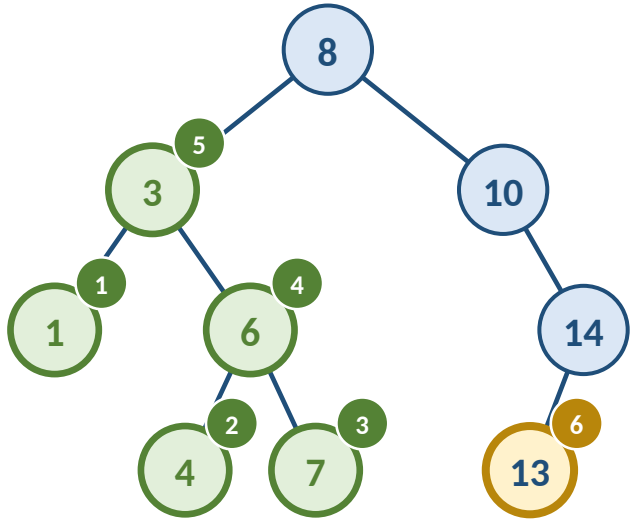


Ausgabe: `"1 3 4 6 7 8 "`

rekursiv:

```
in(l) + item + " " + in(r)
```

`postorder` links · rechts · Wurzel



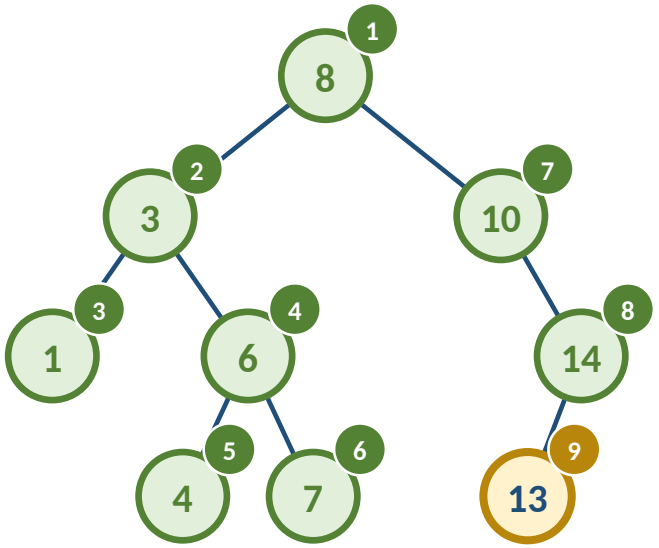
Ausgabe: `"1 4 7 6 3 13 "`

rekursiv:

```
post(l) + post(r) + item + " "
```

Traversierungen: pre-, in- und postorder (3/3)

`preorder` Wurzel · links · rechts

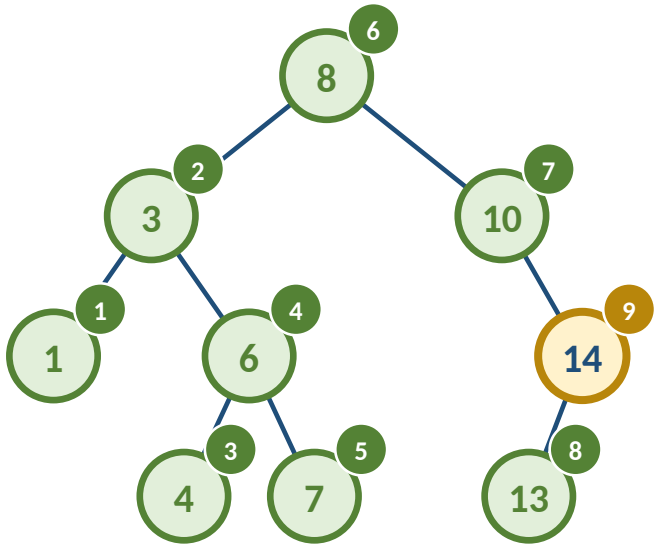


Ausgabe: `"8 3 1 6 4 7 10 14 13 "`

rekursiv:

```
item + " " + pre(l) + pre(r)
```

`inorder` links · Wurzel · rechts

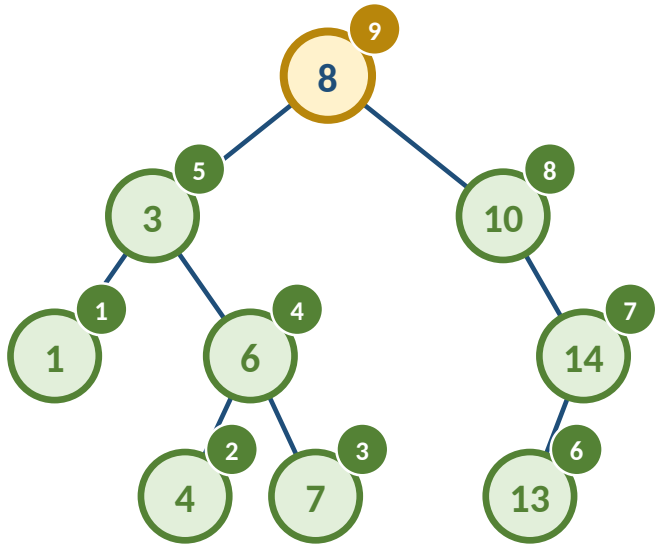


Ausgabe: `"1 3 4 6 7 8 10 13 14 "`

rekursiv:

```
in(l) + item + " " + in(r)
```

`postorder` links · rechts · Wurzel

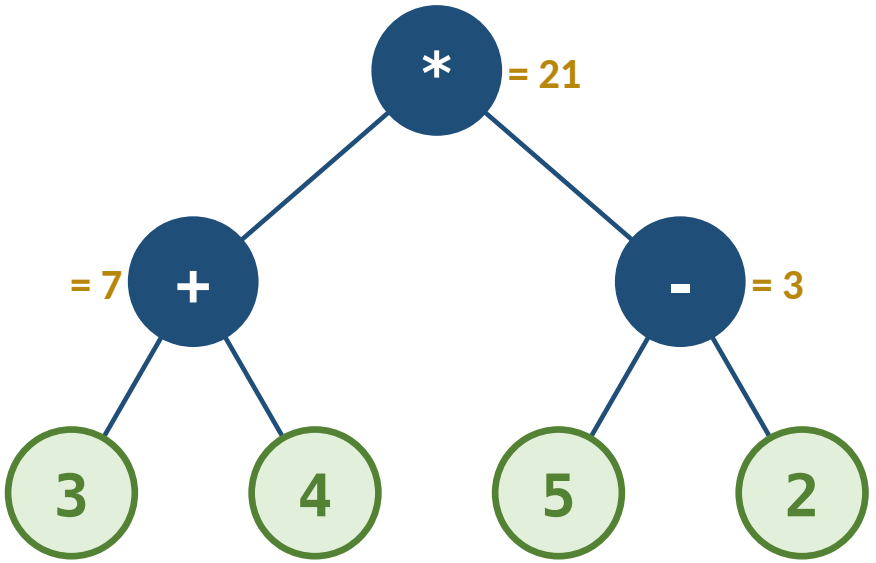


Ausgabe: `"1 4 7 6 3 13 14 10 8 "`

rekursiv:

```
post(l) + post(r) + item + " "
```

Termbaum: (3 + 4) * (5 – 2)



innere Knoten: Operatoren · Blätter: Zahlen (als Zeichenketten)

Traversierung	Ausgabe	Schreibweise
`preorder`	`"* + 3 4 - 5 2 "`	Präfix
`inorder`	`"3 + 4 * 5 - 2 "`	**Infix – Klammern fehlen!**
`postorder`	`"3 4 + 5 2 - * "`	Postfix – wie der Rechner aus Welt 4

Auswerten = postorder denken: erst beide Teilbäume ausrechnen, dann den Operator der Wurzel anwenden: 7 * 3 = 21.

Achtung Abi: "12" → 12 nur mit erlaubten Operationen: charAt, (int)-Wert des Zeichens – kein Integer.parseInt.

Experte L5.1: `berechne(t: Binärbaum vom Inhaltstyp Zeichenkette): Ganzzahl` – Basisfall Blatt: Zahl; sonst Operator anwenden.

Online-IDE: Level 5.1 – class Baum

IDE-Level 5.1 · `class Baum` Testklasse L51BaumTest

Basis ★

`basis_…`

```
anzahlKnoten(b): Ganzzahl
preorder(b) · inorder(b) · postorder(b)
```

Erweiterung ★★

`erweiterung_…`

```
hoehe(b): Ganzzahl
anzahlBlaetter(b): Ganzzahl
summe(b): Ganzzahl
```

Experte ★★★

`experte_…`

```
berechne(t): Ganzzahl    // Termbaum
spiegeln(b): nichts      // links ↔ rechts
```

Datei: AQ_W5_L5_Wald_Geruest.txt · Bibliothek „Niedersachsen“ aktivieren!

Baumschule (vorgegeben)

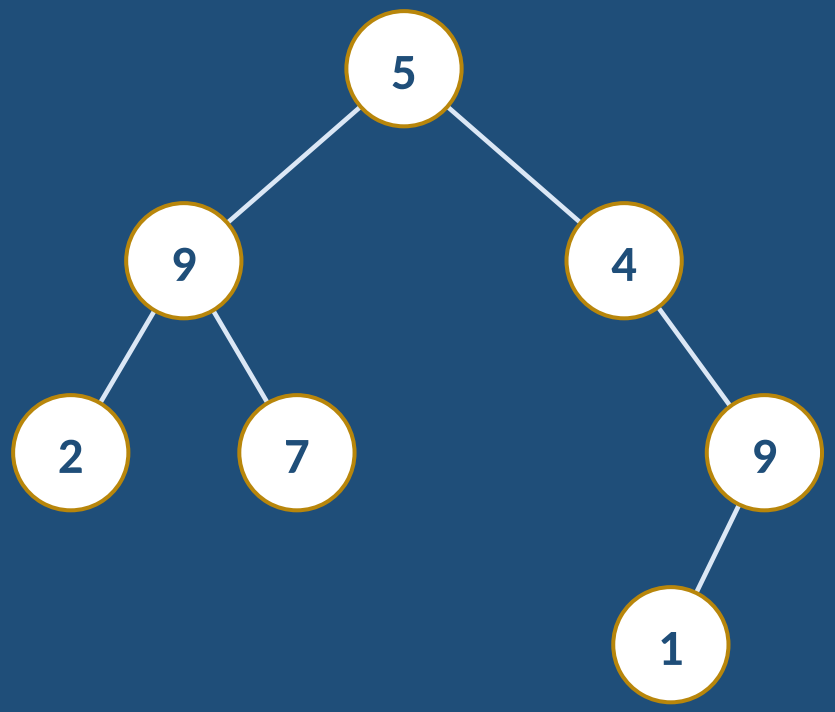
```
beispielBST()
zahlenbaum()
entartet()
termbaum()
monsterquiz()
leer() · blatt(x)
```

jede Methode liefert einen NEUEN Baum

Abi-Check Bäume

- ✓ Basisfall: if (b.isEmpty())
- ✓ beide Teilbäume rekursiv
- ✓ Cast: (Integer) b.getItem()
- ✓ Rückgabe in jedem Pfad

Exit-Ticket DS 31: zahlenbaum()



zahlenbaum() – kein Suchbaum, die 9 kommt doppelt vor

PAPIER · 5 min

1 Gib preorder, inorder und postorder an (Format wie in der IDE).

2 Nenne die Blätter, die Höhe und die Ebene des Knotens 1.

3 Schreibe die Codezeile, die an die rechte 9 ihr linkes Kind 1 hängt.

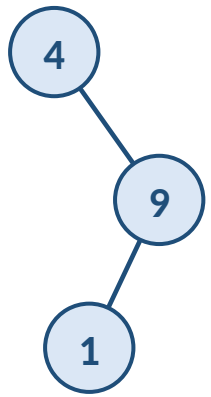
Selbstcheck: Jede Traversierung enthält alle 7 Werte – genau einmal pro Knoten.
Sicherung: Warm-up W5-03 · W5-04 · W5-02

Papier-Warm-up: Wie hoch ist der Baum?

PAPIER · ohne Gerät · 10 min

Papier-Warm-up W5-05 · Baum C = rechter Teilbaum des zahlenbaums

Baum C




```
public static int hoehe(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    int l = hoehe(b.getLeft());
    int r = hoehe(b.getRight());
    if (l > r) {
        return l + 1;
    }
    return r + 1;
}
```

a) Rekursionstabelle für hoehe(C): Tiefe, Aufruf, Rückgabewert (Erstaufruf = Tiefe 0, leere Teilbäume als hoehe(leer)).

b) Wie oft wird hoehe bei n Knoten aufgerufen? Begründe. Und beim zahlenbaum?

So gehst du vor

1. **Abstieg**
   Aufrufe in der Reihenfolge ihres Beginns – jede Ebene tiefer: Tiefe + 1
2. **Basisfall**
   hoehe(leer) liefert 0 – auch das ist ein Aufruf!
3. **Aufstieg**
   Rückgaben von unten nach oben eintragen
4. **Zählen**
   Wie viele Zeilen hat deine Tabelle?

Achtung Abi: Der Erstaufruf hat Tiefe 0.

Das Muster: erst isEmpty, dann links und rechts

```
public static int f(BinTree b) {
    if (b.isEmpty()) {
        return NEUTRAL;      // Basisfall
    }
    int l = f(b.getLeft());   // links
    int r = f(b.getRight());  // rechts
    return KOMBINIERE(b.getItem(), l, r);
}
```

Vertrauen: Nimm an, f liefert für die beiden Teilbäume schon das richtige Ergebnis. Du musst nur noch den Basisfall und das Kombinieren festlegen.

Muster für alle Level 5.1 und 5.2

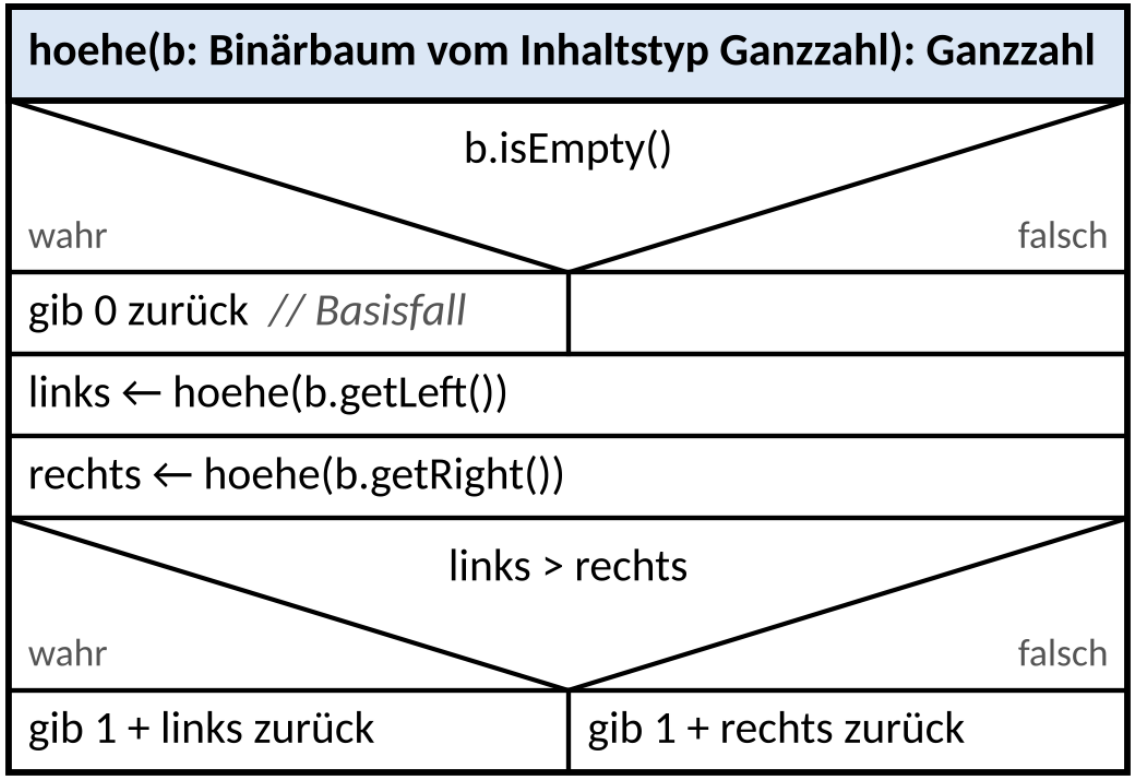
Methode	leerer Baum	kombiniere	beispielBST	zahlenbaum
`anzahlKnoten`	`0`	`1 + l + r`	**9**	**7**
`summe`	`0`	`item + l + r`	**66**	**37**
`hoehe`	`0`	`1 + max(l, r)`	**4**	**4**
`anzahlBlaetter`	`0`	`Blatt? 1 : l + r`	**4**	**3**
`maximum`	`−1`	`max(item, l, r)`	**14**	**9**
`anzahlGroesser(b, 4)`	`0`	`(item>4) + l + r`	**6**	**4**

Falle: isLeaf() als einziger Basisfall. Die 4 im zahlenbaum hat nur ein Kind – der Aufruf für den leeren linken Teilbaum fragt isLeaf() → IllegalStateException (Warm-up W5-06).

hoehe: Java-Code und rekursives Struktogramm

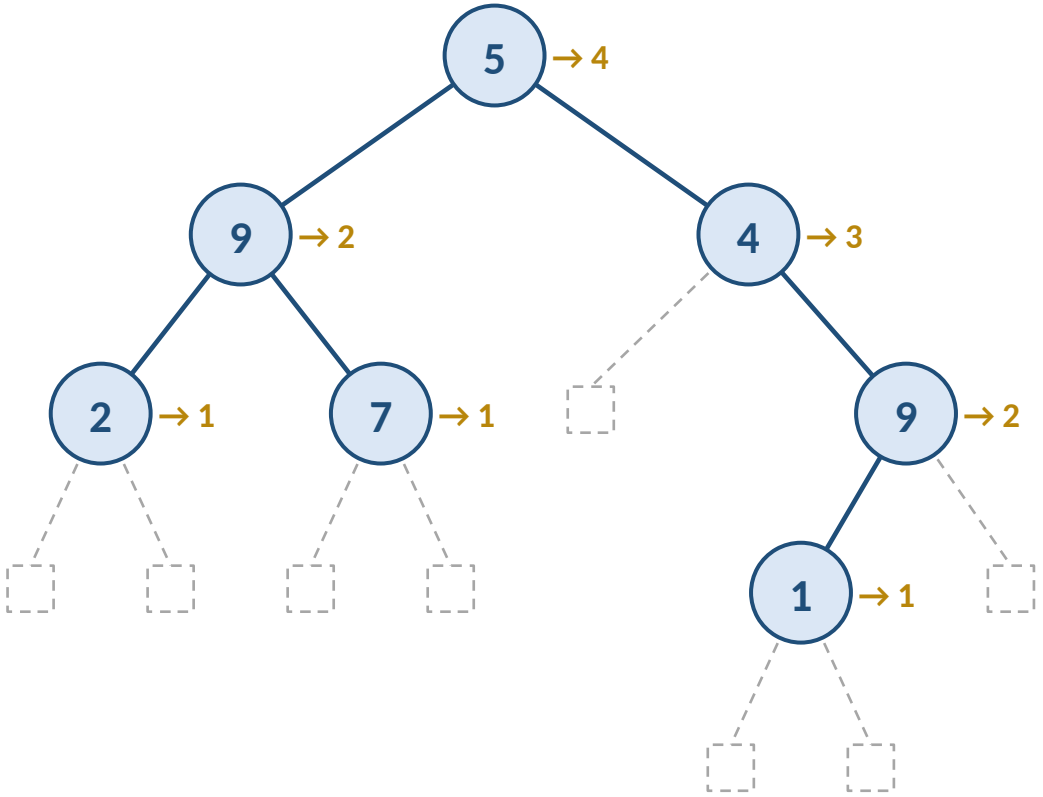
```java
public static int hoehe(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    int links = hoehe(b.getLeft());
    int rechts = hoehe(b.getRight());
    if (links > rechts) {
        return 1 + links;
    }
    return 1 + rechts;
}
```

```java
public static int anzahlKnoten(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    return 1 + anzahlKnoten(b.getLeft())
             + anzahlKnoten(b.getRight());
}
```



Struktogramm nach EH §5: Kopf mit Signatur, Zuweisung mit ←, der Selbstaufruf ist eine ganz normale Anweisung. „gib … zurück“ beendet den Aufruf.

Rekursionstabelle und Aufrufbaum: hoehe(zahlenbaum())



Kästchen = Aufruf mit leerem Baum → 0 · gold = Rückgabe

15 Aufrufe (= 2 · 7 + 1) · größte Tiefe 4

Tiefe	Aufruf	links	rechts	Rückgabe
0	**hoehe(5)**	2	3	**4**
1	**hoehe(9)**	1	1	**2**
2	**hoehe(2)**	0	0	**1**
3	hoehe(leer)			0
3	hoehe(leer)			0
2	**hoehe(7)**	0	0	**1**
3	hoehe(leer)			0
3	hoehe(leer)			0
1	**hoehe(4)**	0	2	**3**
2	hoehe(leer)			0
2	**hoehe(9)**	1	0	**2**
3	**hoehe(1)**	0	0	**1**
4	hoehe(leer)			0
4	hoehe(leer)			0
3	hoehe(leer)			0

Sammeln in einer DynArray: sammleInorder

```
public static DynArray sammleInorder(BinTree b) {
    DynArray d = new DynArray();
    sammleInorder(b, d);       // EINE Reihung für alle
    return d;
}

public static void sammleInorder(BinTree b, DynArray d) {
    if (b.isEmpty()) {
        return;
    }
    sammleInorder(b.getLeft(), d);
    d.append(b.getItem());     // Object hinein
    sammleInorder(b.getRight(), d);
}
```

Reihenfolge = Traversierung: append vor den Aufrufen → preorder, dazwischen → inorder, danach → postorder.

`sammleInorder(zahlenbaum())`

2	9	7	5	4	1	9
0	1	2	3	4	5	6

`sammleInorder(beispielBST())`

1	3	4	6	7	8	10	13	14
0	1	2	3	4	5	6	7	8

sortiert – Inorder eines Suchbaums!

`blaetter(zahlenbaum())`

2	7	1

Blattwerte von links nach rechts

Fehlerbild: new DynArray() im rekursiven Aufruf – jeder Aufruf füllt seine eigene Reihung, das Ergebnis geht verloren (Warm-up W5-08 c).

Online-IDE: Level 5.2 – class Rekursiv

IDE-Level 5.2 · `class Rekursiv` Testklasse L52RekursivTest

Basis ★

`basis_…`

```
enthaelt(b, x): Wahrheitswert
maximum(b): Ganzzahl   // leer: −1
anzahlGroesser(b, grenze): Ganzzahl
```

Erweiterung ★★

`erweiterung_…`

```
sammleInorder(b): DynArray
blaetter(b): DynArray
anzahlAufEbene(b, ebene): Ganzzahl
```

Experte ★★★

`experte_…`

```
pfadZu(b, x): Zeichenkette  // "LR"
istGleich(a, b): Wahrheitswert
```

Alle Methoden rekursiv · Werte in Zahl-Bäumen: Ganzzahlen ≥ 0

Tipps

`anzahlAufEbene`

Ebene 0 = diese Wurzel; sonst in den Kindern Ebene − 1 zählen.

`pfadZu`

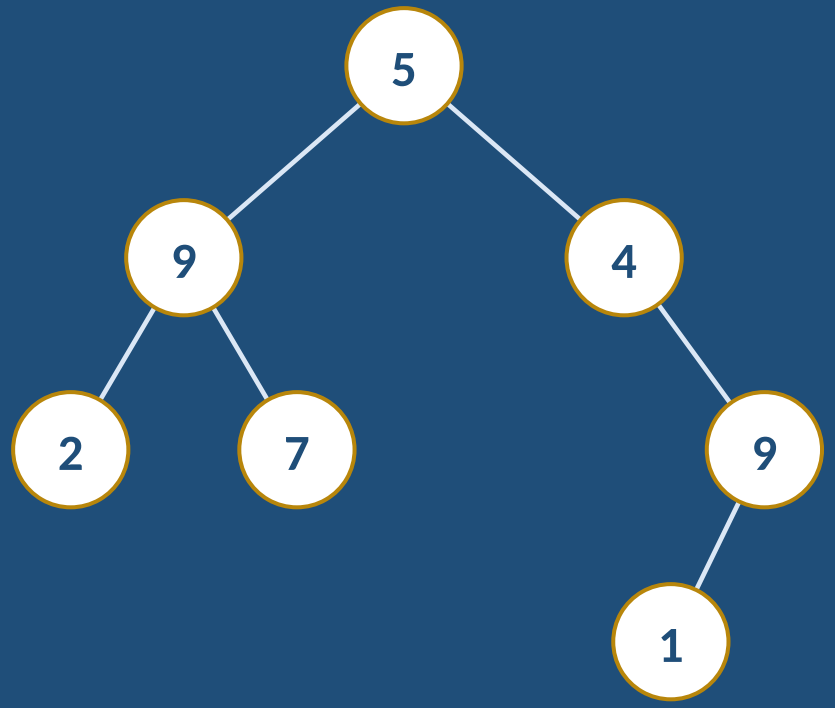
"L" bzw. "R" vor den Weg aus dem Teilbaum hängen; "-" heißt: nicht gefunden.

`istGleich`

Beide leer? Einer leer? Sonst Wert und beide Teilbäume vergleichen.

Integer vergleichen: erst casten – `(Integer) b.getItem() == x`

Exit-Ticket DS 32: Rekursion am zahlenbaum()



PAPIER · 8 min

1 Gib an: `anzahlAufEbene(b, 2)`, `pfadZu(b, 7)`, `pfadZu(b, 1)`

2 Implementiere `maximum(b: Binärbaum vom Inhaltstyp Ganzzahl): Ganzzahl` (leerer Baum → −1) – handschriftlich.

3 Wie viele Aufrufe macht deine Methode für den zahlenbaum?

Papier-Compiler: Deine Lösung zu 2 abtippen – läuft sie sofort?
Sicherung: Warm-up W5-07 · W5-08 · W5-06

Kür: Der Akinator rät dein Monster

PAPIER · ohne Gerät · 10 min

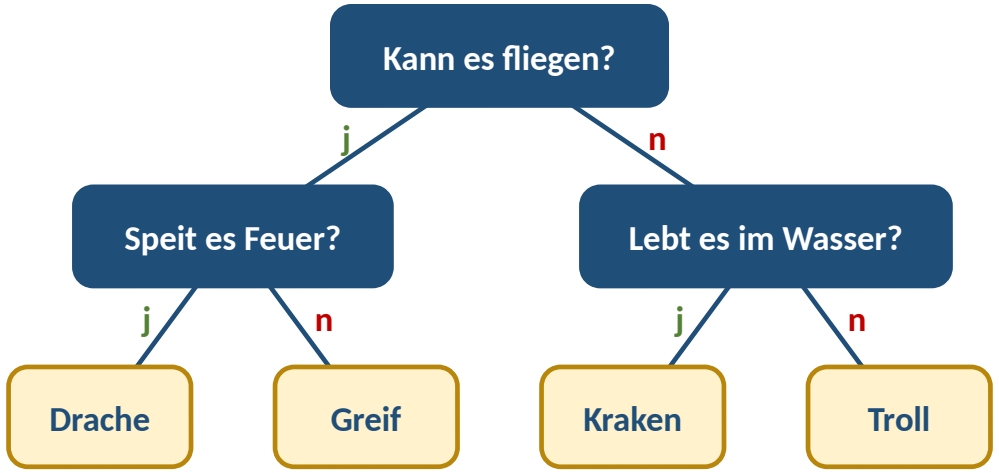
Papier-Warm-up W5-06: Der Ast bricht

```
1  public static int summe(BinTree b) {
2      if (b.isLeaf()) {
3          return b.getItem();
4      }
5      return b.getItem() + summe(b.getLeft())
                          + summe(b.getRight());
6  }
```

a) Welche Zeilen akzeptiert der Compiler nicht? Korrigiere sie.

b) Aufruf mit dem zahlenbaum: Gib die Aufrufe bis zum Laufzeitfehler an. Ursache?

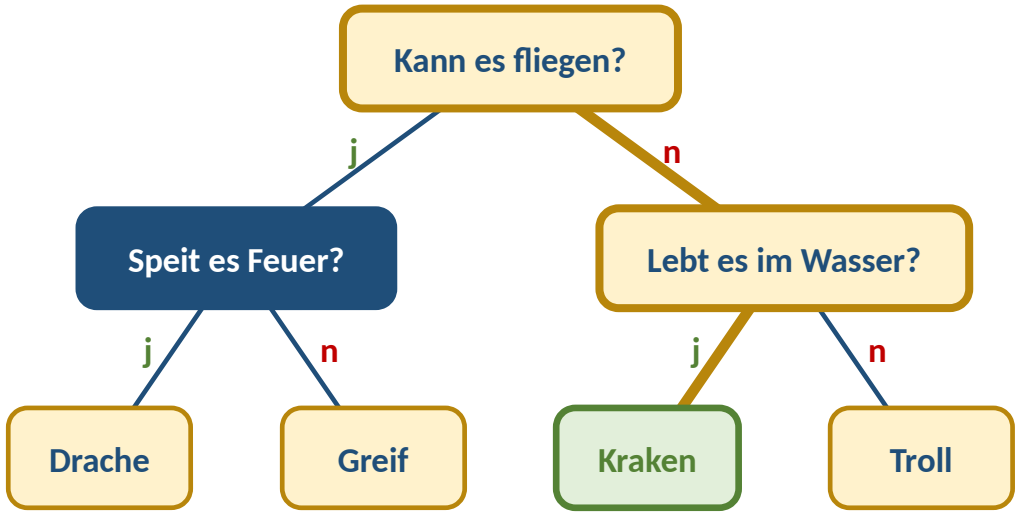
c) Korrigiere die Methode für jeden Baum (auch den leeren). summe(zahlenbaum) = ?



monsterquiz() · ja → links, nein → rechts · Fragen innen, Monster in den Blättern

Unplugged: Rate mit!
Was liefert raten("jn")? raten("nn")? raten("j")? raten("nnj")? Denk dir ein Monster aus, das der Baum nicht kennt – mit welcher Frage unterscheidest du es?

raten: Von der Wurzel bis zum Blatt – Antwort für Antwort



raten("nj"): n → „Lebt es im Wasser?“, j → **Kraken**

`"jj"` → Drache	`"jn"` → Greif	`"nj"` → Kraken
`"nn"` → Troll	`"j"` → ?	`"nnj"` → Troll

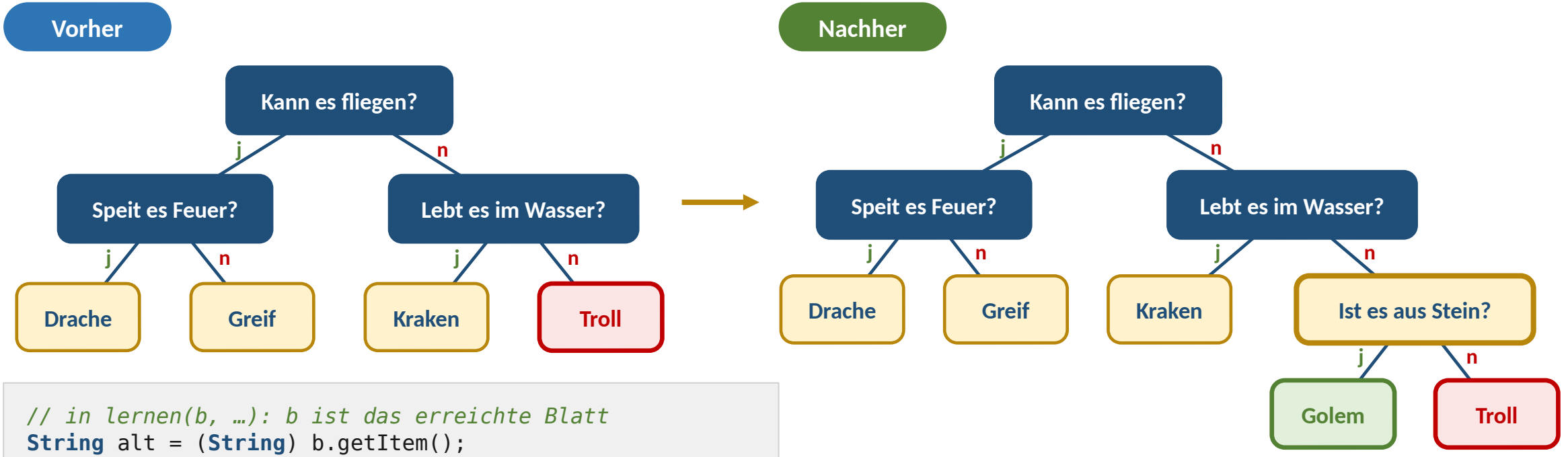
zu wenige Antworten → "?" · überzählige werden ignoriert

```
public String raten(String antworten) {
    return raten(baum, antworten);
}

private static String raten(BinTree b, String antworten) {
    if (b.isEmpty()) {
        return "?";
    }
    if (b.isLeaf()) {
        return (String) b.getItem();      // Monster gefunden
    }
    if (antworten.length() == 0) {
        return "?";            // Frage, aber keine Antwort mehr
    }
    String rest = antworten.substring(1, antworten.length());
    if (antworten.charAt(0) == 'j') {
        return raten(b.getLeft(), rest);
    }
    return raten(b.getRight(), rest);
}
```

Rest der Antworten: substring(1, antworten.length()) – erst isEmpty, dann isLeaf.

lernen: Der Baum wächst um eine Frage




```
// in lernen(b, …): b ist das erreichte Blatt
String alt = (String) b.getItem();
b.setItem(frage);                // Blatt wird Frageknoten
if (jaFuerNeu) {
    b.setLeft(new BinTree(neu));
    b.setRight(new BinTree(alt));
} else {
    b.setLeft(new BinTree(alt));
    b.setRight(new BinTree(neu));
}
```

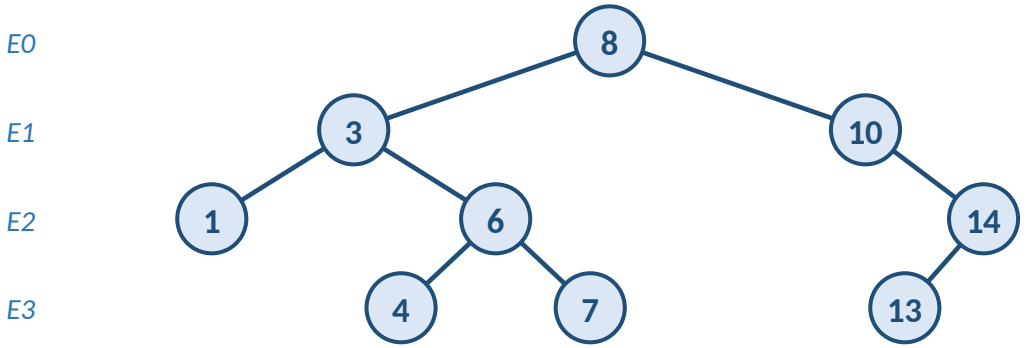
`lernen("nn", "Golem", "Ist es aus Stein?", true)`

	vorher	nachher
`anzahlMonster()`	4	5
`anzahlFragen()`	3	4
`raten("nnj")`	Troll	Golem
`raten("nn")`	Troll	?

Level-Order: ebenenweise mit einer Schlange

```
public static String levelorder(BinTree b) {
    String text = "";
    Queue q = new Queue();
    if (!b.isEmpty()) {
        q.enqueue(b);
    }   // → Tabellenzeile (Startwerte)
    while (!q.isEmpty()) {
        BinTree k = (BinTree) q.dequeue();
        text = text + k.getItem() + " ";
        if (!k.getLeft().isEmpty()) {
            q.enqueue(k.getLeft());
        }
        if (!k.getRight().isEmpty()) {
            q.enqueue(k.getRight());
        }   // → Tabellenzeile
    }
    return text;
}
```

Schlange vom Inhaltstyp Binärbaum: In der Schlange stehen ganze Teilbäume – beim Entnehmen Cast (BinTree).



Nr	entnommen	Schlange (vorne … hinten)	text
0	–	8	""
1	8	3 10	"8 "
2	3	10 1 6	"8 3 "
3	10	1 6 14	"8 3 10 "
4	1	6 14	"8 3 10 1 "
5	6	14 4 7	"8 3 10 1 6 "
6	14	4 7 13	"8 3 10 1 6 14 "
7	4	7 13	"8 3 10 1 6 14 4 "
8	7	13	"8 3 10 1 6 14 4 7 "
9	13	–	"8 3 10 1 6 14 4 7 13 "

Online-IDE: Level 5.3 – class Akinator · Exit-Ticket

IDE-Level 5.3 · `class Akinator` Testklasse L53AkinatorTest

Basis ★

`basis_…`

```
c Akinator(start)
raten(antworten): Zeichenkette
```

Erweiterung ★★

`erweiterung_…`

```
lernen(antworten, neu, frage, jaFuerNeu)
anzahlMonster() · anzahlFragen()
```

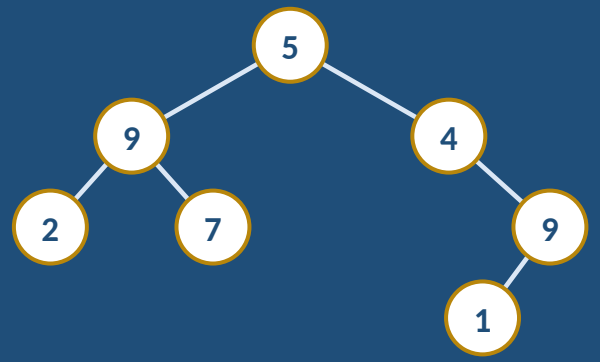
Experte ★★★

`experte_…`

```
static levelorder(b): Zeichenkette
```

Vorgegeben: c Akinator(b) und getBaum() · Hilfsmethoden private static

Exit-Ticket DS 33



1 Gib die Level-Order von zahlenbaum() an.

2 Notiere nach jedem dequeue den Inhalt der Schlange.

3 Wie viele Monster hat ein Akinator mit 7 Fragen?

Sicherung: Warm-up W5-09

Einstieg: Die Item-Datenbank braucht Ordnung

PAPIER · ohne Gerät · 10 min

Papier-Warm-up W5-10: Die Item-Datenbank wächst

Füge die Item-Nummern nacheinander in einen leeren Baum ein:

`8, 3, 10, 1, 6, 14, 4, 7, 13`

Regel: ab der Wurzel – kleiner → links, größer → rechts, bis ein Platz frei ist.

a) Zeichne den Baum. Kennst du ihn?

b) Füge anschließend 5 und 12 ein. Gib jeweils den Suchweg an und markiere die neuen Knoten.

c) Gib die Inorder an und begründe, warum sie sortiert ist. Was passiert beim Einfügen von 6?

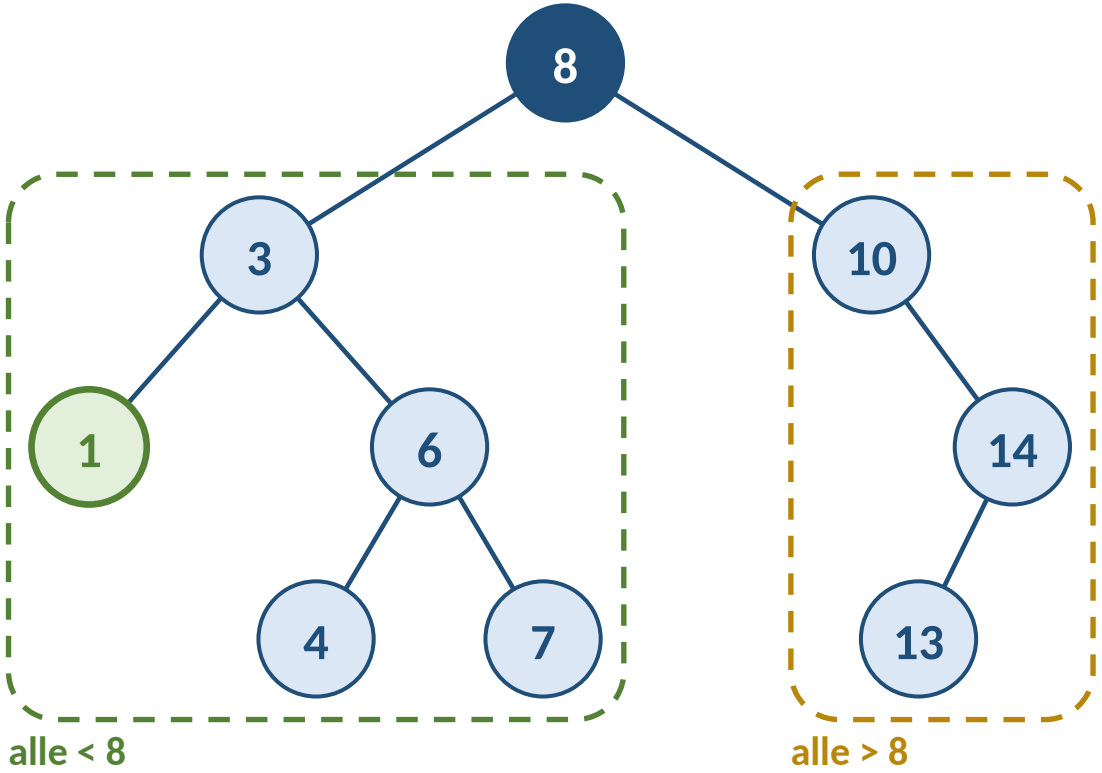


Spielmodul Item-Datenbank

Jedes Item hat eine Nummer. Spieler tragen Hunderte Items – die Suche nach einer Nummer muss schnell gehen.

Leitfrage: Wie viele Knoten muss ich höchstens ansehen – wenn der Baum geordnet ist?

Suchbaum: links kleiner, rechts größer – überall



Ordnung (BST): Für jeden Knoten gilt: alle Werte im linken Teilbaum < Wert < alle Werte im rechten Teilbaum. Duplikate werden ignoriert.

inorder liefert die Werte sortiert:

1	3	4	6	7	8	10	13	14

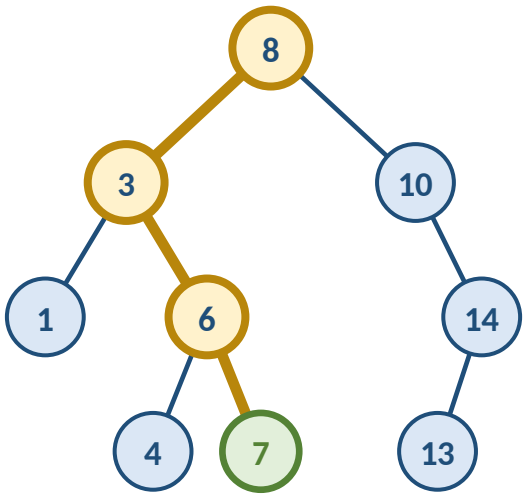
Nicht nur Eltern und Kind: auch 7 (ganz unten) muss kleiner als 8 sein.

minimum(beispielBST()) = 1 · Maximum ganz rechts: 14

Suchen: Bei jedem Knoten nur EIN Teilbaum

```java
public static boolean suche(BinTree b, int x) {
    if (b.isEmpty()) {
        return false;        // nicht da
    }
    int wert = (Integer) b.getItem();
    if (x == wert) {
        return true;
    }
    if (x < wert) {
        return suche(b.getLeft(), x);
    }
    return suche(b.getRight(), x);
}
```

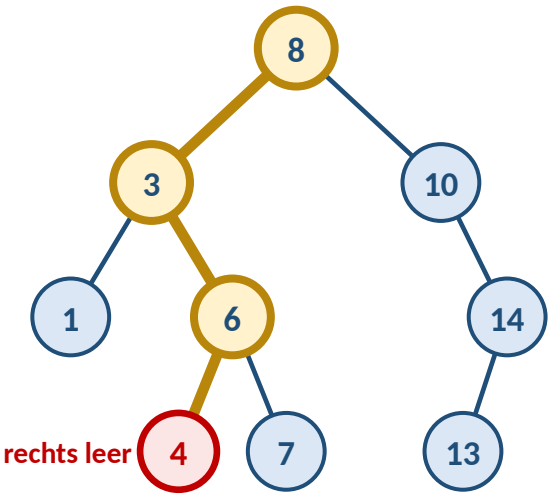
suche(b, 7)



Weg: 8 3 6 7

vergleiche = 4 · true

suche(b, 5)



Weg: 8 3 6 4 leer

vergleiche = 4 · false

Aufwand: höchstens so viele Vergleiche wie die Höhe – hier 4 statt 9 Knoten.

Iterativ (Erweiterung): Schleife solange nicht leer – b = b.getLeft() bzw. b.getRight().

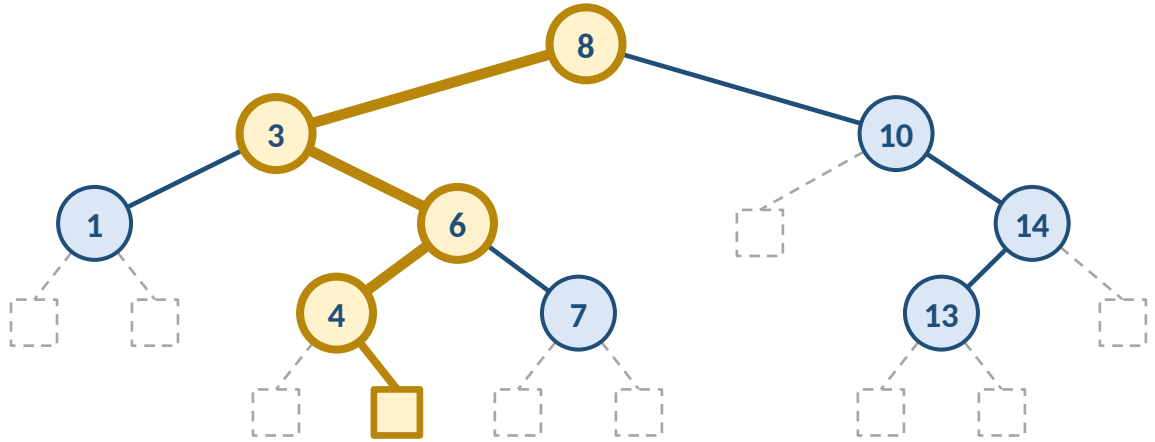
Einfügen: suchen, bis ein leerer Teilbaum kommt

```
public static void einfuegen(BinTree b, int x) {
    if (b.isEmpty()) {
        b.setItem(x);     // leer -> wird Blatt
        return;
    }
    int wert = (Integer) b.getItem();
    if (x < wert) {
        einfuegen(b.getLeft(), x);
    } else if (x > wert) {
        einfuegen(b.getRight(), x);
    }                     // Duplikat: nichts tun
}
```

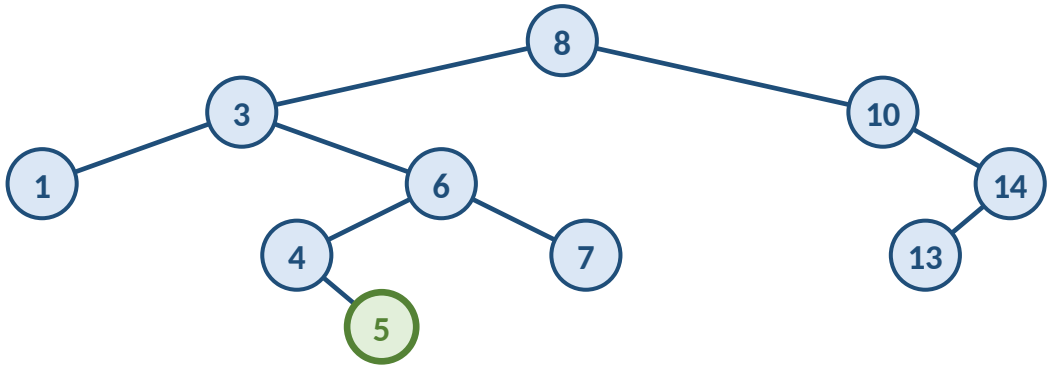
Trick: setItem auf dem leeren Teilbaum macht ihn zum Blatt – er hängt schon am Elternknoten.

Duplikat: einfuegen(b, 8) ändert nichts. Achtung: Die Höhe wächst hier von 4 auf 5.

Vorher: einfuegen(b, 5)

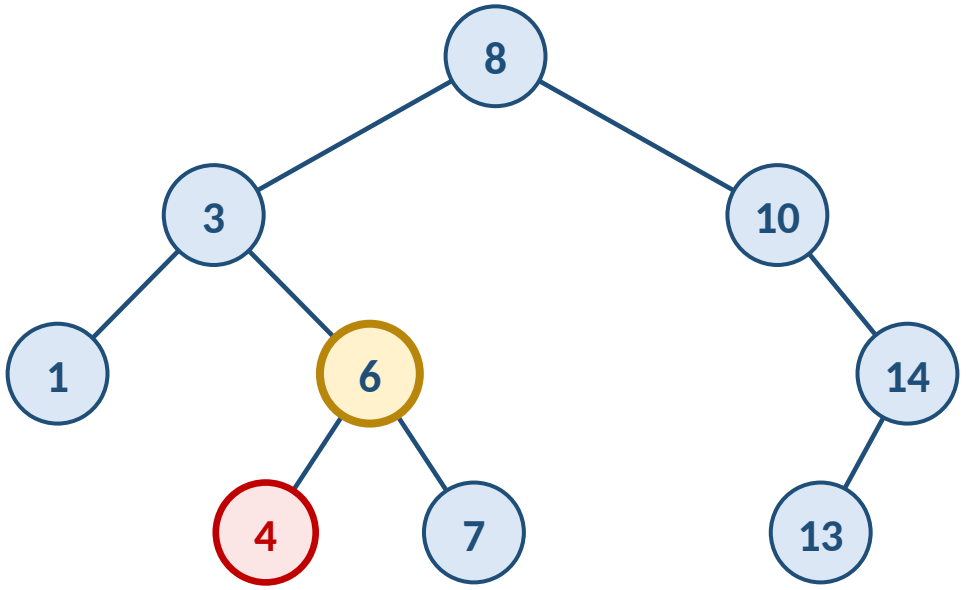


Nachher

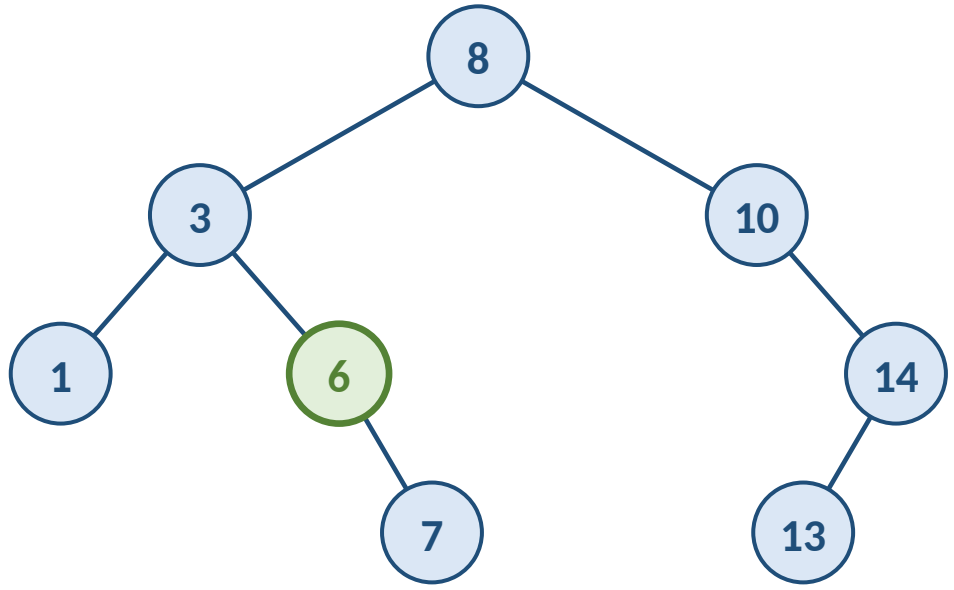


Löschen 1/3: Ein Blatt entfernen (lösche 4)

Vorher



Nachher

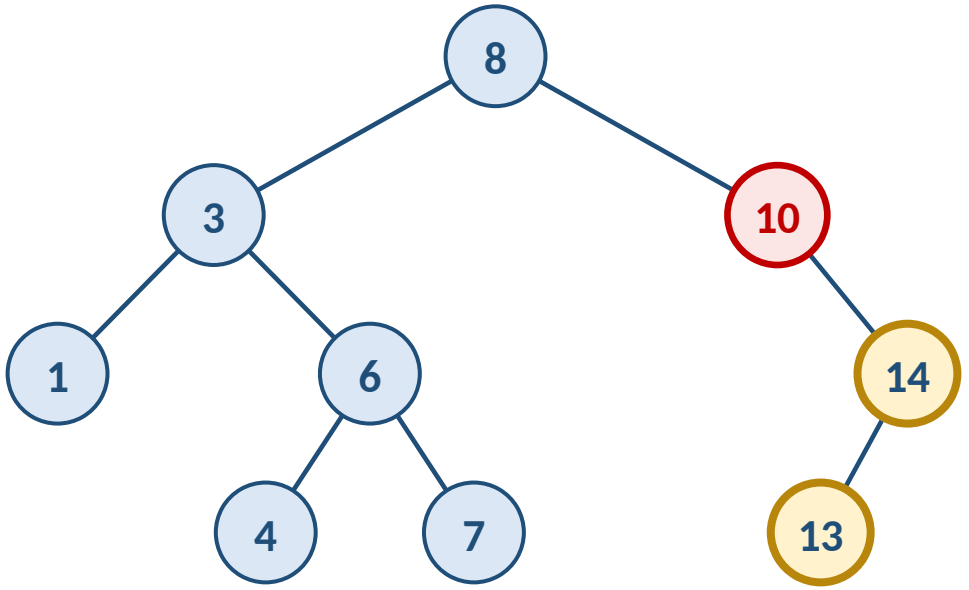


Fall 1 – Blatt: einfach entfernen; der Elternknoten (6) bekommt dort einen leeren Teilbaum. In Java: `k.setEmpty()`

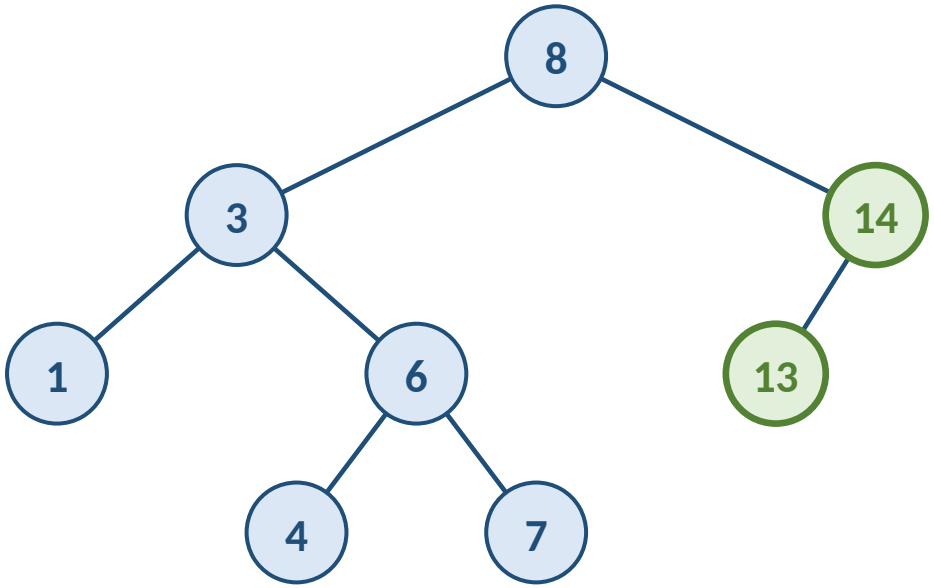
Probe – inorder danach:
`"1 3 6 7 8 10 13 14"` **sortiert** ✓

Löschen 2/3: Knoten mit einem Kind (lösche 10)

Vorher



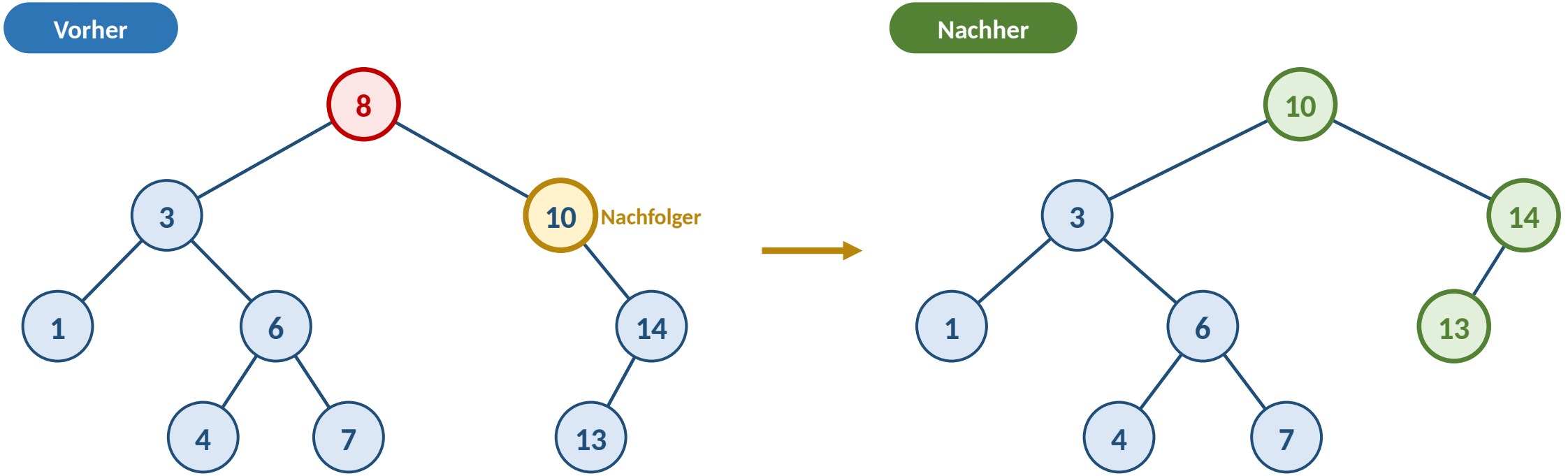
Nachher



Fall 2 – ein Kind: das einzige Kind rückt mit seinem ganzen Teilbaum an die frei gewordene Stelle. Die Ordnung bleibt: alles rechts von 8 bleibt rechts von 8.

Probe – inorder danach:
`"1 3 4 6 7 8 13 14"` **sortiert ✓**

Löschen 3/3: Knoten mit zwei Kindern (lösche 8)



Fall 3 – zwei Kinder: Nachfolger = kleinster Wert im rechten Teilbaum (einmal rechts, dann ganz links): 10. Seinen Wert in den Knoten kopieren, dann den Nachfolger löschen (Fall 1 oder 2). Alternativ: Vorgänger = größter links (7).

Probe – inorder danach:
`"1 3 4 6 7 10 13 14"` **sortiert ✓**

Online-IDE: Level 5.4 – class Suchbaum

IDE-Level 5.4 · `class Suchbaum` Testklasse L54SuchbaumTest

Basis ★

`basis_…`

```
suche(b, x): Wahrheitswert
minimum(b): Ganzzahl   // leer: −1
einfuegen(b, x): nichts
```

Erweiterung ★★

`erweiterung_…`

```
sucheIterativ(b, x): Wahrheitswert
vergleiche(b, x): Ganzzahl
baueAus(r: Reihung von Ganzzahl): BinTree
```

Experte ★★★

`experte_… (DS 35)`

```
istBST(b) · ausgeglichenAus(sortiert)
nachfolger(b, x): Ganzzahl  // −1: keiner
```

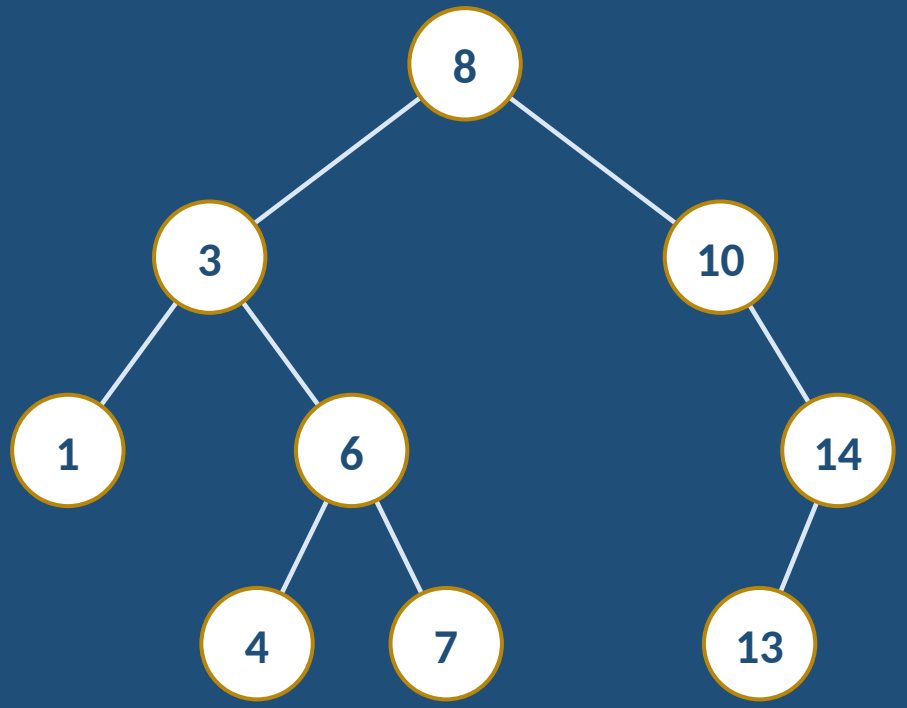
suche und einfuegen rekursiv, nur EIN Teilbaum je Schritt · Löschen: nur grafisch

Testwerte beispielBST()

`vergleiche(b, 7)`	4
`vergleiche(b, 5)`	4
`nachfolger(b, 8)`	10
`nachfolger(b, 7)`	8
`nachfolger(b, 14)`	−1
`minimum(b)`	1

Häufige Fehler: beide Teilbäume durchsuchen (kein BST-Vorteil) · b = new BinTree(x) statt b.setItem(x) · Duplikat doppelt eingefügt.

Exit-Ticket DS 34: beispielBST()



PAPIER · 6 min

1 Gib den Suchweg und die Anzahl der Vergleiche für suche(b, 13) an.

2 Füge 9 ein. Wo landet sie? Begründe mit dem Suchweg.

3 Lösche 6 aus dem ursprünglichen Baum mit dem Nachfolger. Zeichne den neuen Baum.

Selbstcheck: Ist die inorder deines neuen Baums sortiert?
Sicherung: Warm-up W5-11 · W5-12

Einstieg: Die Item-Datenbank wird langsam

PAPIER · ohne Gerät · 10 min

Papier-Warm-up W5-13: Krumm oder buschig?

Die Items kommen sortiert an. Füge in einen leeren Suchbaum ein:

`1, 2, 3, 4, 5`

a) Zeichne den entstehenden Baum und den Baum ausgeglichenAus für r = {1, 2, 3, 4, 5}. Gib jeweils die Höhe an.

b) Die maximale Anzahl der Vergleiche ist die Höhe. Gib sie für n = 7, 15, 31 und 1023 an – entartet und ausgeglichen. Was bewirkt eine Verdopplung von n?



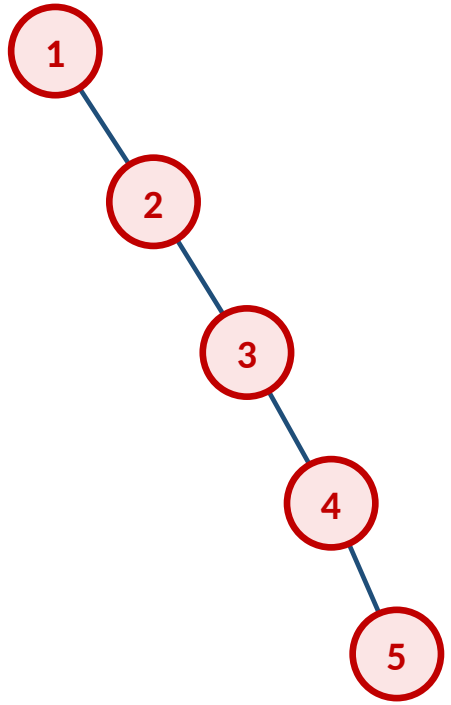
Bug-Report der Spieler

„Seit dem Update dauert die Item-Suche ewig!“ – Das Update hat alle Items nach Nummer sortiert eingetragen.

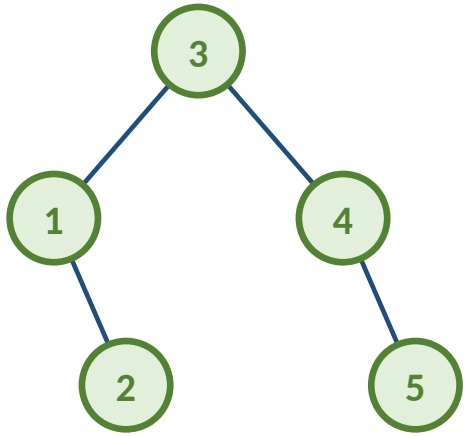
Leitfrage: Hängt der Aufwand der Suche von der Anzahl der Items ab – oder von der Form des Baums?

Entartet oder ausgeglichen? Die Form entscheidet

entartet() · Höhe 5



ausgeglichen · Höhe 3



gleiche Werte 1 … 5, andere Einfügereihenfolge (3, 1, 4, 2, 5)

höchstens nötige Vergleiche (= Höhe)

n Items	ausgeglichen	entartet
7	3	7
15	4	15
31	5	31
1 023	10	1 023
2 047	11	2 047
4 095	12	4 095
1 000 000	20	1 000 000

Entartet = Liste: jeder Knoten hat höchstens ein Kind – Suchen wie in einer Reihung.

Verdopplung von n → nur 1 Vergleich mehr. Höhe ≈ $\log_2 n$ statt n.

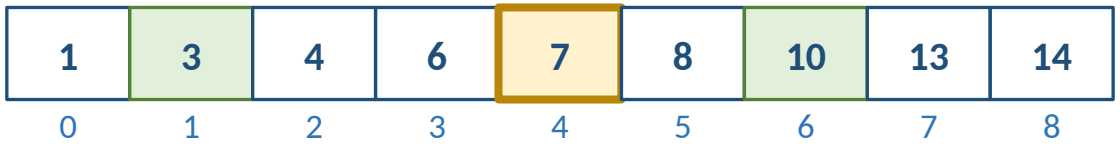
ausgeglichenAus: Die Mitte wird Wurzel – rekursiv

```java
public static BinTree ausgeglichenAus(int[] sortiert) {
    return ausgeglichenAus(sortiert, 0, sortiert.length - 1);
}

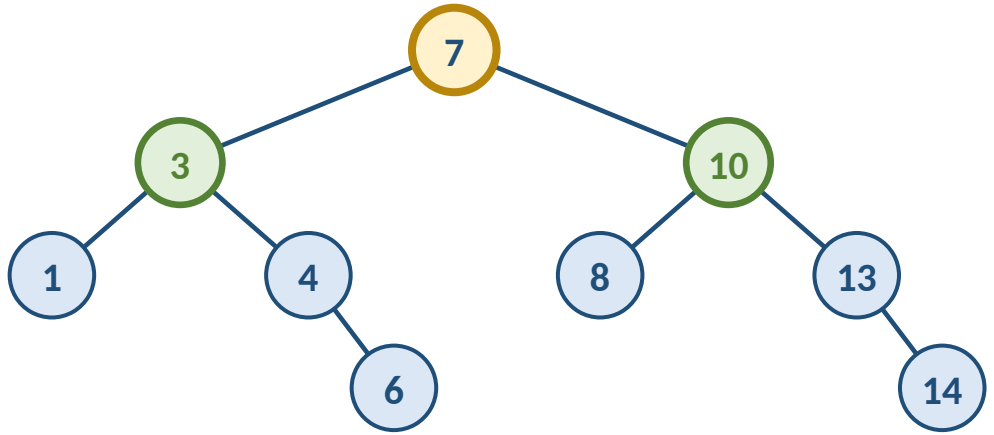
public static BinTree ausgeglichenAus(int[] r,
                                      int lo, int hi) {
    if (lo > hi) {
        return new BinTree();     // Bereich leer
    }
    int mitte = (lo + hi) / 2;
    BinTree b = new BinTree(r[mitte]);
    b.setLeft(ausgeglichenAus(r, lo, mitte - 1));
    b.setRight(ausgeglichenAus(r, mitte + 1, hi));
    return b;
}
```

Teile und herrsche (Welt 3): mitte = (lo + hi) / 2 ganzzahlig – wie bei der binären Suche. Links von der Mitte entsteht der linke, rechts der rechte Teilbaum; leerer Bereich (lo > hi) → leerer Baum.

sortiert = inorder von beispielBST()

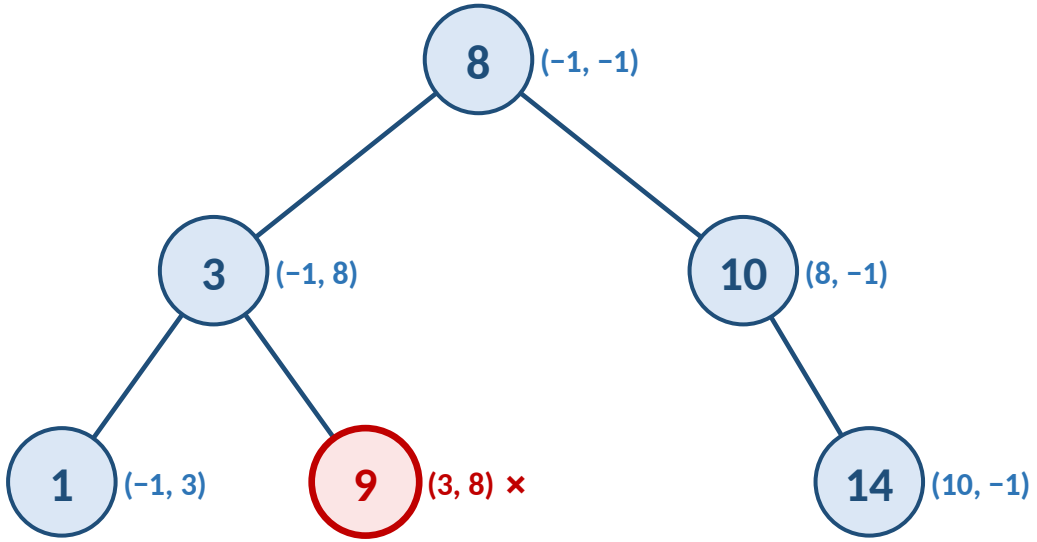


Mitte: `[0..8] → 7` **dann:** `[0..3] → 3` · `[5..8] → 10`



Höhe 4 = minimal: 3 Ebenen fassen nur 7 Knoten, für 9 braucht man 4.

Die isBST-Falle: Nur die Kinder prüfen reicht nicht



neben jedem Knoten: (min, max), exklusiv · −1 = keine Grenze

```java
public static boolean istBST(BinTree b) {
    return istBST(b, -1, -1);   // keine obere Grenze
}

public static boolean istBST(BinTree b, int min, int max) {
    if (b.isEmpty()) {
        return true;
    }
    int wert = (Integer) b.getItem();
    if (wert <= min) {
        return false;
    }
    if (max != -1 && wert >= max) {
        return false;
    }
    return istBST(b.getLeft(), min, wert)
        && istBST(b.getRight(), wert, max);
}
```

Jeder Knoten passt zu seinen Kindern (3 < 9 ✓) – trotzdem kein BST: 9 liegt links von 8. suche(b, 9): 8 → 10 → leer → false, obwohl die 9 im Baum steht.

Grenzen wandern mit: nach links wird wert die neue Obergrenze, nach rechts die neue Untergrenze.

Level 5.4 Experte · Exit-Ticket DS 35

IDE · Experte ★★★

```
istBST(b): Wahrheitswert
ausgeglichenAus(sortiert): BinTree
nachfolger(b, x): Ganzzahl
```

- istBST mit Grenzen – nicht nur Kinder
- mitte = (lo + hi) / 2, leerer Bereich → leerer Baum
- nachfolger: kleinster Wert > x, sonst −1
- Papier-Compiler: eine Methode erst auf Papier

Testklasse L54SuchbaumTest · experte_…

PAPIER · 6 min

1 Ein ausgeglichener Suchbaum enthält 1 000 Items. Wie viele Vergleiche braucht eine Suche höchstens? Und bei 2 000?

2 Zeichne ausgeglichenAus({2, 4, 6, 8, 10, 12, 14}).

3 Begründe in einem Satz, warum istBST Grenzen braucht.

Sicherung: Warm-up W5-14
Boss-Training: W5-05 · W5-07 · W5-08 · W5-13 · W5-14

Strategie: Eine unbekannte Baumart erschließen

1



Definition lesen

Begriffe markieren: Was ist ein Knoten, was steht darin, was ist erlaubt?

BST: „links kleiner, rechts größer“

2



Beispiel von Hand

Kleines Beispiel zeichnen und jede Operation einmal durchspielen.

BST: 8, 3, 10 einfügen

3



Invariante formulieren

Was gilt IMMER – vor und nach jeder Operation? Ein Satz mit „für jeden Knoten …“.

BST: für jeden Knoten links < w < rechts

4



Auf BinTree abbilden

Was ist getItem()? Basisfall isEmpty()? Welche Teilbäume braucht die Operation?

BST: suche geht nur in EINEN Teilbaum

Achtung Abi: Unbekannte Bäume (Heap, Huffman, B-Baum, Trie …) werden im Material vollständig definiert – lesen schlägt auswendig lernen.

Boss 5: Die drei Baumhüter



Station A

Max-Heap

Ordnung erkennen, Heap von Hand, Vergleich mit dem Suchbaum

≈ 12 BE · unbekannte Baumart



Station B

Huffman-Baum

Baum nach Verfahren aufbauen, codieren und decodieren

≈ 12 BE · unbekannte Baumart



Station C

B-Baum

Ordnungsprinzip, Einfügen mit Teilung, Suchaufwand

≈ 12 BE · unbekannte Baumart



Wähle 2 von 3
Stationen in 60 min · 24 BE



Papier, ohne Gerät
Alle Definitionen stehen im Material



Zusätzlich im Lab
Online-Boss „Die drei Baumhüter“ · 25 min

Vorbereitung: Warm-up W5-05, W5-07, W5-08, W5-13, W5-14 · AB3 „Unbekannte Bäume“ · Abi-Check isEmpty()